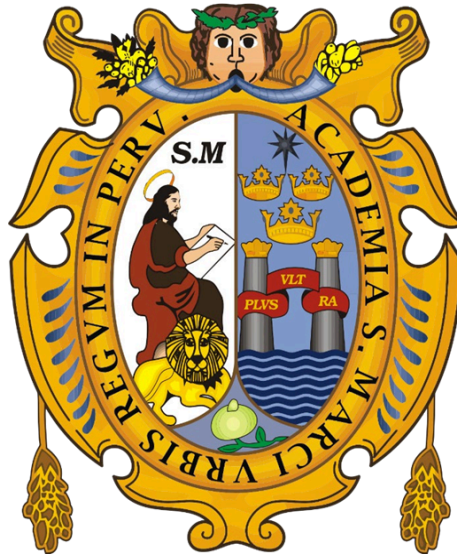


UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA
E.P DE INGENIERÍA DE SOFTWARE



Gestión de la configuración de software

Plan de gestión de la configuración de software

DOCENTE:

Dra. Marlene Reyes

ALUMNOS:

Bohorquez Huaranga Kevin Rhamses

León Chacón Gerardo Raúl

Lopez Leonardo

Rojas Rojas Max

Sanchez Gotea Edu Joseph

Sevan Reyes David

Taco Zavala Miguel Angel

Vera Rodriguez Cristian Leonardo

Plan de gestión de la configuración de software

Patitas Sanas

Versión 1.0.0

Historia de revisiones

Fecha	Versión	Descripción	Autor
07/05/2026	1.0.0v	Creación inicial del documento	Edu Sanchez
25/05/2026	1.1.0v	Actualización mayor por incorporación del Sprint 2 (Reportería PDF) y nuevos Elementos de Configuración (ECS).	Edu Sanchez / Lopez Leonardo
02/06/2026	1.2.0v	Cierre del Sprint 3: Implementación del sistema de alertas y notificaciones automatizadas para recordatorios de citas con nuevos ECS.	Edu Sanchez / León Gerardo
13/06/2026	1.3.0v	Actualizando los ECS por finalización de sprint 4	Edu Sanchez / Taco Miguel

Índice

1. Introducción y Alcance	3
2. Organización y Responsabilidades	4
2.1. Estructura organizacional	4
2.2. Roles del equipo	4
3. Actividades de Gestión de Configuración	6
3.1. Identificación de la Configuración	6
3.1.1. Catálogo de ECS del Proyecto Patitas Sanas	7
3.1.2. Nomenclatura de Versiones	8
3.1.3 Estructura del Repositorio	8
4. Cronograma y Líneas Base (Baselines)	11
5. Herramientas y Entorno	13
5.8. Comunicación y Reportes de Estado	15
Referencias	16

1. Introducción y Alcance

1.1. Introducción

El presente documento define el Plan de Gestión de la Configuración de Software (PGCS) para el proyecto Patitas Sanas, un sistema de gestión clínica veterinaria.

En un entorno de desarrollo colaborativo es fundamental establecer mecanismos que garanticen la integridad, trazabilidad y control de cambios en los productos de software generado teniendo eso en cuenta este plan detalla las normativas, herramientas y procesos que permitirán que el código fuente, la documentación y los entornos de ejecución evolucionen de forma organizada.

1.2. Propósito

El propósito de este plan es proporcionar una base sólida para el Control de los Elementos de Configuración de software (ECS). Se busca minimizar los riesgos derivados de cambios no autorizados, asegurar que el equipo trabaje sobre versiones validadas y facilitar la auditoria del ciclo de vida del proyecto.

1.3. Alcance

1.3.1. Desarrollo de Software (Código Fuente)

- **Backend:** Implementación de la lógica de negocio utilizando el framework FastAPI en Python. Abarca los módulos de registro de mascotas, historiales clínicos, gestión de veterinarios, especialidades, turnos de trabajo, citas y autenticación por roles. A esto se suman los módulos desarrollados en el ciclo de sprints: un servicio de generación y exportación de reportes clínicos y administrativos en PDF con endpoints protegidos y extracción de datos relacionales; un sistema de notificaciones automáticas por correo con tareas en segundo plano que disparan recordatorios 2 horas antes de cada cita; un módulo financiero para el registro de ingresos, gastos operativos, nóminas y cálculo de Ganancia Neta; y una capa analítica que expone los agregados estadísticos consumidos por el dashboard gerencial.
- **Frontend:** Implementación de la interfaz de usuario utilizando el framework React + Vite. Abarca los mismos módulos del núcleo que el backend, complementados con las interfaces desarrolladas en el ciclo de sprints: componentes con botones de descarga segura para los reportes PDF; un dashboard de Business Intelligence con gráficos de barras, diagramas sectoriales y tarjetas de KPIs para ingresos estimados y tasa de asistencia; y un panel de control financiero para la gestión del flujo de caja y consulta del balance neto.

1.3.2. Gestión de Datos

Diseño y mantenimiento de una base de datos relacional en MySQL. Esto incluye los scripts de creación de esquemas (DDL), la carga de datos semilla (DML) y la gestión de migraciones para asegurar la evolución estructural del sistema.

1.3.3. Infraestructura y Despliegue

Despliegue del servidor y de la base de datos en Railway. Despliegue del frontend en Vercel

1.3.4. Documentación

Elaboración del Documento de Requerimientos del Software (SRS), el Plan de Gestión de Configuración y manuales detallados de usuario y despliegue además de un seguimiento del proyecto mediante la app Jira.

1.4. Glosario

- **ECS (Elemento de Configuración de Software):** Cualquier pieza de software, documentación o datos que deba ser controlada.
- **CCB (Configuration Control Board):** Comité de Control de Cambios encargado de evaluar y aprobar modificaciones.
- **SemVer:** Estándar de Versionado Semántico utilizado para la nomenclatura de versiones.
- **Baseline (Línea Base):** Especificación o producto que ha sido revisado formalmente y sobre el cual se basará el desarrollo posterior.

2. Organización y Responsabilidades

2.1. Estructura organizacional

El proyecto estará conformado por estudiantes de la carrera de ingeniería de software de séptimo ciclo de la Facultad de Ingeniería de Sistemas e Informática (FISI) de la Universidad Nacional Mayor de San Marcos (UNMSM) que se encargaran de las diferentes etapas del desarrollo de software, incluyendo planificación, análisis, diseño, implementación, pruebas y mantenimiento.

La estructura organizacional del proyecto se presenta de la siguiente manera

- Cliente / Usuario Final
- Jefe de Proyecto
- Analista de Sistemas
- Equipo de Desarrollo
- Equipo de Pruebas y Calidad
- Soporte Técnico

El flujo de comunicación y supervisión será jerárquico, donde el Jefe de Proyecto coordinará las actividades generales y supervisará el cumplimiento de los objetivos establecidos.

2.2. Roles del equipo

ROL	FUNCION	RESPONSABILIDAD
Jefe de Proyecto - Vera Rodriguez	- Dirigir y supervisar el desarrollo general del proyecto. - Coordinar las actividades del equipo de trabajo. - Gestionar el cumplimiento del cronograma y objetivos. - Aprobar entregables y validar avances del proyecto	- Garantizar el correcto desarrollo del proyecto. - Resolver problemas organizacionales y técnicos. - Mantener comunicación con el cliente y el equipo.
Gestor de Configuración de Software (CS) - Edu Sanchez	- Administrar el control de versiones del proyecto. - Supervisar cambios en el software y documentación. - Gestionar repositorios y herramientas de integración.	- Garantizar la integridad y trazabilidad de los cambios. - Mantener actualizado el repositorio del proyecto. - Coordinar el control de configuración del software.
Bibliotecario - Max Rojas	- Organizar y administrar la documentación del proyecto. - Mantener el registro de versiones de documentos. - Gestionar el almacenamiento de manuales, reportes y archivos técnicos.	- Garantizar la disponibilidad y actualización de la documentación. - Controlar el acceso y distribución de documentos oficiales. - Mantener respaldo de la

			información del proyecto.
Control de comité de cambios	Líder del Comité- Kevin Bohorquez	- Coordinar las reuniones del comité de cambios. - Evaluar solicitudes de modificación del proyecto. - Supervisar el impacto de los cambios aprobados.	- Autorizar cambios importantes en el sistema. - Garantizar que las modificaciones no afecten la estabilidad del proyecto.
	Arquitecto técnico - Taco Miguel	- Diseñar la arquitectura técnica del sistema. - Definir estándares tecnológicos y estructurales. - Evaluar impactos técnicos de cambios solicitados.	- Garantizar la escalabilidad y mantenibilidad del sistema. - Supervisar la correcta integración de componentes.
	Responsable de calidad - León Gerardo	- Supervisar el cumplimiento de estándares de calidad. - Planificar y validar pruebas del sistema. - Elaborar reportes de control de calidad.	- Garantizar la calidad del software antes de cada entrega. - Verificar el cumplimiento de requisitos funcionales y no funcionales.
	Desarrollador Backend - Sevan David	- Implementar la lógica de negocio del sistema. - Desarrollar APIs y servicios backend. - Gestionar la conexión con la base de datos.	- Garantizar el correcto funcionamiento del servidor. - Optimizar rendimiento y seguridad del sistema.
	Desarrollador Frontend - Lopez Leonardo	- Diseñar e implementar interfaces gráficas. - Mejorar la experiencia de usuario. - Integrar frontend con servicios backend.	- Garantizar accesibilidad y usabilidad del sistema. - Mantener coherencia visual de la aplicación.

3. Actividades de Gestión de Configuración

3.1. Identificación de la Configuración

La Identificación de la Configuración es el proceso mediante el cual se seleccionan, nombran, describen y clasifican formalmente todos los elementos del proyecto que serán objeto de control y seguimiento. Esta actividad establece la estructura base sobre la que operan los demás procesos de gestión de configuración.

Para el sistema Patitas Sanas, la identificación de la configuración abarca los artefactos producidos en el diseño, desarrollo, pruebas y despliegue del sistema de gestión clínica veterinaria, incluyendo tanto el software funcional como la documentación de soporte.

3.1.1. Catálogo de ECS del Proyecto Patitas Sanas

#	Categoría	Elemento de Configuración (ECS)	Identificador
1	Código Fuente – Backend	Módulo de gestión de mascotas (registro, historial clínico)	SCI-BE-001
2	Código Fuente – Backend	Módulo de veterinarios y especialidades	SCI-BE-002
3	Código Fuente – Backend	Módulo de turnos de trabajo	SCI-BE-003
4	Código Fuente – Backend	Módulo de citas y agenda	SCI-BE-004
5	Código Fuente – Backend	Módulo de autenticación y roles	SCI-BE-005
6	Código Fuente – Backend	API REST / controladores	SCI-BE-006
7	Código Fuente - Backend	Servicio/Controlador de generación de reportes clínicos en PDF	SCI-BE-007
8	Código Fuente - Backend	Módulo de notificaciones	SCI-BE-008
9	Código Fuente - Backend	Pruebas automatizadas del backend (pytest)	SCI-BE-009
10	Código Fuente – Frontend	Pantalla de registro y perfil de mascotas	SCI-FE-001
11	Código Fuente – Frontend	Panel de agenda y citas	SCI-FE-002
12	Código Fuente – Frontend	Panel de administración de veterinarios	SCI-FE-003
13	Código Fuente – Frontend	Panel de turnos	SCI-FE-004
14	Código Fuente - Frontend	Componente de vista previa y descarga de PDFs	SCI-FE-005

15	Código Fuente - Frontend	Dashboard analítico con métricas y gráficos	SCI-FE-006
16	Código Fuente - Frontend	Pruebas automatizadas del frontend	SCI-FE-007
17	Base de Datos	Scripts DDL (creación de esquema)	SCI-DB-001
18	Base de Datos	Scripts DML (datos semilla / seed)	SCI-DB-002
19	Base de Datos	Scripts de migración SQL	SCI-DB-003
20	Infraestructura	Variables de entorno (.env.example)	SCI-INF-001
21	Infraestructura	Configuración del servicio de email (email-service-config.md)	SCI-INF-002
22	Infraestructura	Configuración de SonarQube	SCI-INF-003
23	Infraestructura	Configuración de despliegue (Railway, Vercel)	SCI-INF-004
24	Documentación	Documento de Requerimientos del Software (SRS)	SCI-DOC-001
25	Documentación	Plan de Gestión de Configuración	SCI-DOC-002
26	Documentación	Manuales de usuario y despliegue	SCI-DOC-003
27	Documentación	Línea base del proyecto	SCI-DOC-004
28	Documentación	Definición Ambiente Controlado	SCI-DOC-005
29	Documentación	Registro de versiones	SCI-DOC-006
30	Documentación	Gestión de cambios	SCI-DOC-007
31	Documentación	Informe Parcial de SCM	SCI-DOC-008
32	Documentación	Changelog.md: registrar historial de versiones	SCI-DOC-009
33	Documentación	Guías de calidad de código	SCI-DOC-010
34	Documentación	Registro de análisis de sonarqube y evidencias por sprint	SCI-DOC-011

3.1.2. Nomenclatura de Versiones

Todo ECS sujeto a control de versiones adoptará el estándar de Versionado Semántico (SemVer), ampliamente aceptado en la industria del software. El esquema es el siguiente:

MAJOR (X)	MINOR (Y)	PATCH (Z)
compatibilidad con versiones anteriores. Cambio que rompe	Funcionalidad nueva compatible hacia atrás.	Corrección de errores.

Formato completo: v{MAJOR}.{MINOR}.{PATCH}

Ejemplo de Versión	Significado
v0.1.0	Primera versión funcional interna (módulo de mascotas completado).
v0.2.0	Se agrega el módulo de veterinarios y especialidades.
v0.2.1	Corrección de bug en el filtro de especialidades.
v1.0.0	Primera línea base formal entregada al cliente / evaluador.
v1.1.0	Módulo de turnos de trabajo integrado.

3.1.3 Estructura del Repositorio

El repositorio de Patitas Sanas en GitHub tendrá la siguiente estructura de directorios para garantizar que cada ECS esté ubicado en un lugar predecible y controlado:

Ruta en el Repositorio	Contenido (ECS almacenados)
/backend/	Código fuente del servidor (SCI-BE-001 al SCI-BE-007)
/frontend/	Código fuente de la interfaz (SCI-FE-001 al SCI-FE-005)
/veterinariaDump	Dump de la base de datos, estructura y registros (SCI-DB-001 y SCI-DB-002)
/docs/	Documentación del proyecto (SCI-DOC-001 al SCI-DOC-003)
/.env.example	Plantilla de variables de entorno (SCI-INF-002)

/README.md	Guía rápida del proyecto y de contribución
/CHANGELOG.md	Registro histórico de cambios por versión

3.2. Control de la Configuración

El control de configuración del proyecto “Patitas Sanas” tiene como objetivo garantizar que los cambios realizados sobre el sistema sean gestionados de manera organizada, controlada y trazable durante el desarrollo del software. Para ello, se utilizará Git y GitHub como herramientas de control de versiones y administración del repositorio central. La gestión de configuración permitirá mantener la integridad del sistema y asegurar que las modificaciones realizadas sean registradas y validadas adecuadamente.

Todo cambio realizado en el proyecto deberá seguir un proceso de control previamente establecido. Las modificaciones podrán originarse por corrección de errores, implementación de nuevas funcionalidades, mejoras del sistema o ajustes solicitados durante el desarrollo. Cada solicitud de cambio será evaluada por el Comité de Control de Cambios (CCB), el cual determinará su aprobación y prioridad.

3.2.1 Estrategia de Ramas

Para organizar el desarrollo del proyecto, se utilizará la siguiente estrategia de ramas:

Tipo de Rama	Patrón de Nombre	Propósito
Principal	main	Código en producción. Solo acepta merges desde release/*.
Integración	develop	Rama de integración continua. Aquí se fusionan todas las features completadas.
Nueva funcionalidad	feature/{descripcion-corta}	Desarrollo de un ECS nuevo.
Versión de lanzamiento	release/v{X}.{Y}.{Z}	Preparación de una línea base.

3.2.2 Flujo de Control de Cambios

El flujo de control de cambios definido para el proyecto será el siguiente:

1. Registro y evaluación de la solicitud de cambio.

2. Creación de una rama feature/* a partir de develop.
3. Desarrollo e implementación de los cambios correspondientes.
4. Registro de cambios mediante commits en el repositorio.
5. Creación de un Pull Request hacia la rama develop.
6. Revisión y validación del cambio por parte del Líder Técnico o responsable de QA.
7. Integración del cambio mediante merge hacia develop.
8. Creación de una rama release/v* cuando las funcionalidades alcancen estabilidad.
9. Integración final de la versión aprobada hacia la rama master.

Con el fin de mantener el control y la estabilidad del proyecto, no se permitirán cambios directos sobre las ramas master y develop. Todo cambio deberá integrarse mediante Pull Requests y pasar por un proceso de revisión antes de su aprobación.

3.2.3 Convención de Commits

Los commits realizados en el proyecto deberán seguir una nomenclatura estándar con el fin de facilitar la trazabilidad y comprensión de los cambios realizados. Para ello, se utilizarán las siguientes etiquetas:

Tipo	Descripción	Ejemplo
feat	Nueva funcionalidad.	feat(citas): agregar filtro por especialidad
fix	Corrección de un bug.	fix(turnos): corregir solapamiento de horarios
docs	Cambios solo en documentación.	docs(readme): actualizar instrucciones de despliegue
refactor	Refactorización sin cambio de comportamiento.	refactor(veterinarios): simplificar lógica de búsqueda
test	Agregar o modificar pruebas.	test(mascotas): agregar prueba de registro duplicado
chore	Tareas de mantenimiento.	chore(deps): actualizar dependencias npm

Ejemplos:

- ❖ feat: agregar módulo de citas
- ❖ fix: corregir validación de login
- ❖ docs: actualizar documentación del backend

3.3. Contabilidad del Estado de Configuración

La Contabilidad del Estado de Configuración registra y reporta el estado de cada Elemento de Configuración (ECS), las solicitudes de cambio y el avance de las líneas base del proyecto Patitas Sanas.

El responsable de esta actividad es el Gestor de Configuración de Software, quien consolida la información proveniente de Jira y GitHub.

3.3.1. Información a registrar por cada ECS

Atributo	Descripción
Identificador	Código del ECS según el catálogo (p. ej. SCI-BE-001).
Versión vigente	Versión actual bajo el esquema SemVer.
Línea Base	Baseline al que pertenece la versión actual.
Estado	En desarrollo / En revisión / Aprobado / Liberado.
Responsable	Miembro del equipo a cargo del elemento.
Última modificación	Fecha y referencia al commit o Pull Request.

3.3.2. Registros oficiales

Registro	Herramienta	Contenido
Estado de ECS	Hoja en /docs/control/estado_ecs.xlsx	Versión y estado vigente de cada SCI.
Solicitudes de Cambio	Jira (tipo <i>Change Request</i>)	Decisión del CCB y trazabilidad al PR.
Histórico de Versiones	CHANGELOG.md	Cambios consolidados por versión.

3.3.3. Estados de una Solicitud de Cambio

Estado	Descripción
Registrada	Creada en Jira por un miembro del equipo o cliente.
En evaluación	El CCB evalúa impacto y prioridad.
Aprobada	Autorizada por el CCB.
Rechazada	No procede; se documenta la justificación.

3.3.4. Reportes

Reporte	Frecuencia	Contenido
Reporte semanal	Cada viernes	Solicitudes cerradas, abiertas y diferidas; avance hacia la próxima línea base.
Cierre de Línea Base	Al cerrar cada baseline	ECS incluidos, solicitudes integradas, bugs resueltos y diferidos.

3.4. Auditoria y revisiones de configuración

Se realizarán auditorías de configuración antes de la liberación de cada línea base o de una actualización de versión de un componente prioritario. Su objetivo es verificar que la línea base esté compuesta por una colección consistente y bien definida de productos.

3.4.1. Elementos de Configuración bajo Auditoría

Se seleccionarán uno o más ECS de mayor prioridad en la línea base, priorizando aquellos con mayor complejidad técnica, criticidad funcional o mayor número de cambios registrados en Jira.

3.4.2. Agenda de Auditorías

Auditoría	Línea Base	Semana	ECS Prioritarios
AUD-01	LB1 – Core System (Auditoría de Código Heredado)	Semana 3	SCI-BE-005, SCI-BE-002, SCI-FE-003
AUD-02	LB2 – Módulo de Reportería PDF	Semana 5	SCI-BE-007, SCI-FE-005
AUD-03	LB3 – Sistema de Notificaciones y Alarmas	Semana 7	SCI-BE-001, SCI-BE-003, SCI-BE-006
AUD-04	LB4 – Dashboard Analítico y BI	Semana 9	SCI-BE-007, SCI-FE-005, SCI-DB-001
AUD-05	LB5 – Control Financiero y Release Final	Semana 10	Todos los ECS + SCI-DOC-001 al SCI-DOC-003

3.4.3. Conducción y Participantes

Las auditorías serán dirigidas por el SCMR (Edu Sanchez). Los participantes obligatorios son el SCMR y los autores de los ECS a auditar; el Responsable de QA (León Gerardo) y el Líder del CCB (Kevin Bohorquez) participan de forma recomendada.

3.4.4. Documentos Requeridos

- Documentos de Solicitudes de Cambio (SCR) registrados en Jira.
- Reporte de estado de configuración: estado_ecs.xlsx en /docs/control/.
- CHANGELOG.md y Pull Requests cerrados en GitHub.

3.4.5. Reporte de Deficiencias y Acciones Correctivas

Las deficiencias identificadas serán documentadas por los participantes durante la auditoría, registrando el ECS afectado, la descripción del problema y la acción correctiva acordada. Cada deficiencia será cargada como tarea en Jira y monitoreada hasta su cierre por el SCMR.

3.4.6. Criterio de Aprobación

El criterio de aprobación es determinado por el SCMR. Una línea base será aprobada cuando no existan deficiencias críticas pendientes, todos los ECS auditados presenten la versión correcta, y los cambios aprobados por el CCB estén reflejados en el CHANGELOG.md. Las deficiencias menores deben contar con un plan de acción registrado en Jira.

4. Cronograma y Líneas Base (Baselines)

4.1. Definición de Líneas Base (Baselines)

Esta sección documenta las versiones estables congeladas del proyecto Patitas Sanas. Las líneas base constituyen hitos de control técnicos y funcionales que aseguran la integridad del software antes de avanzar hacia nuevas etapas de desarrollo.

Línea Base 1 (v1.0.0) - "Core Clínico Administrativo"

- Origen: Cierre de la Fase Previa del sistema heredado (05/05/2026).
- Identificador en Git: Congelado formalmente bajo el Tag v1.0.0 (Rama main).
- ECS Asegurados: Código fuente Backend completo (SCI-BE-001 al SCI-BE-006), código fuente Frontend (SCI-FE-001 al SCI-FE-004), Base de Datos (SCI-DB-001 y SCI-DB-002), configuración de infraestructura (SCI-INF-002) y documentación inicial (SCI-DOC-001 al SCI-DOC-003).
- Estado: Congelada (Liberada).

Línea Base 2 (v1.1.0) - "Módulo de Reportería PDF"

- Origen: Establecida al finalizar el Sprint 2 (19/05/2026).
- Identificador en Git: Congelado formalmente bajo el Tag v2.0.0.
- ECS Asegurados: Nuevas funciones del servidor de reportes (SCI-BE-007), componentes de la interfaz de usuario (SCI-FE-005) y actualización del Plan de Gestión de Configuración (SCI-DOC-002 v2.0).
- Estado: Congelada (Liberada).

Línea Base 3 (v1.2.0) - "Comunicación Asíncrona y Alertas"

- Origen: Establecida al finalizar el Sprint 3 (02/06/2026).
- Identificador en Git: Planificada bajo el Tag v3.0.0.
- ECS Asegurados: Módulo de automatización de correos en backend, servicios de integración SMTP/API y reglas de negocio para alertas de agenda médica.
- Estado: Planificada.

Línea Base 4 (v1.3.0) - "Capa Analítica y Business Intelligence"

- Origen: Establecida al finalizar el Sprint 4 (16/06/2026).
- Identificador en Git: Planificada bajo el Tag v4.0.0.
- ECS Asegurados: Librerías de gráficos en frontend y endpoints de agregación estadística en el backend.
- Estado: Planificada.

Línea Base 5 (v1.4.0) - "Control Financiero y Release Final"

- Origen: Establecida al finalizar el Sprint 5 (07/07/2026).

- Identificador en Git: Planificada bajo el Tag v5.0.0 (Versión definitiva de producción).
- ECS Asegurados: Módulo financiero completo, manuales de usuario y despliegue actualizados (SCI-DOC-003) y la versión final unificada de todos los componentes de software.
- Estado: Planificada.

4.2. Cronograma de Desarrollo (10 Semanas)

A continuación, se detalla la planificación de los nuevos módulos y funcionalidades avanzadas, dividida en 5 Sprints:

Sprint 1: Documentación y Arquitectura de Integración

- Objetivo: Establecer el punto de partida técnico mediante la auditoría del sistema heredado y la planificación de los nuevos módulos.
- Duración: 2 semanas (28 de abril al 12 de mayo de 2026).
- Actividades realizadas: Revisión exhaustiva del código del proyecto base. Levantamiento, análisis y refinamiento de requerimientos funcionales para la expansión del sistema. Diseño de los nuevos esquemas de base de datos para soportar los módulos financieros y asíncronos. Redacción de la documentación técnica inicial.
- Resultados: Documentación formal aprobada y planificación arquitectónica validada.

Sprint 2: Módulo de Exportación y Reportería PDF

- Objetivo: Habilitar la generación autónoma y descarga segura de documentos clínicos y administrativos en formato PDF.
- Duración: 1 semana (12 de mayo al 19 de mayo de 2026).
- Actividades realizadas: Integración de librerías de generación PDF en el backend. Creación de endpoints protegidos para la descarga de archivos. Diseño de la estructura visual para el reporte de citas diarias. Desarrollo del

algoritmo de extracción de datos relacionales para la exportación completa de historiales clínicos.

- Resultados: Motor de reportes documentales funcional e integrado al panel de administración.

Sprint 3: Sistema de Comunicación Asíncrona y Alarmas

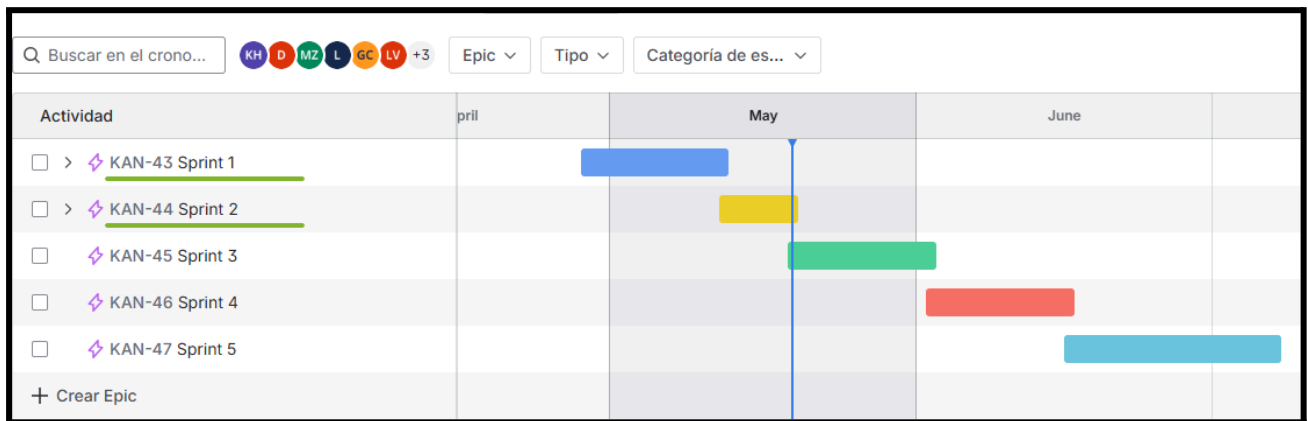
- Objetivo: Reducir la tasa de inasistencia a las citas mediante la implementación de un sistema de notificaciones automáticas al cliente.
- Duración: 2 semanas (19 de mayo al 2 de junio de 2026)
- Actividades realizadas: Configuración del protocolo de envío de correos (SMTP/API) en el servidor. Programación de tareas en segundo plano (cron jobs) enfocadas en el escaneo continuo de la agenda médica. Implementación de la regla de negocio para disparar correos electrónicos recordatorios exactamente 2 horas previas a la cita programada.
- Resultados: Sistema automatizado de alarmas por correo plenamente operativo en el servidor.

Sprint 4: Business Intelligence y Capa Analítica

- Objetivo: Desarrollar una interfaz gerencial que transforme los registros de la base de datos en estadísticas visuales para optimizar la toma de decisiones.
- Duración: 2 semanas (2 de junio al 16 de junio de 2026).
- Actividades realizadas: Integración de librerías de visualización de datos en React. Construcción de gráficos de barras para auditar los servicios médicos más demandados. Diseño de diagramas sectoriales para analizar la distribución de las especies atendidas. Implementación de tarjetas de rendimiento (KPIs) para calcular ingresos estimados y evaluar la tasa de asistencia (citas completadas vs. canceladas).
- Resultados: Dashboard analítico interactivo desplegado en el panel de administración.

Sprint 5: Control Financiero Integral, QA y Despliegue

- **Objetivo:** Centralizar el flujo de caja del negocio en un solo módulo e implementar la versión definitiva del sistema en un entorno de producción.
- **Duración:** 3 semanas (16 de junio al 7 de julio de 2026).
- **Actividades realizadas:** Construcción del módulo de control financiero. Automatización del registro de ingresos, capturando el precio del servicio al marcar una cita como completada. Desarrollo de un panel para la inserción de gastos operativos y el pago de nóminas. Implementación del algoritmo contable para calcular el balance (Ganancia Neta). Ejecución de la fase intensiva de pruebas integrales (QA) y despliegue del sistema final en servidores cloud.
- **Resultados:** Módulo financiero operativo, pruebas finales superadas y sistema desplegado en producción.



5. Herramientas y Entorno

Para garantizar la integridad, trazabilidad y correcta gestión de los Elementos de Configuración (SCI) del proyecto, se ha definido una arquitectura de herramientas basada en estándares de la industria y repositorios distribuidos.

5.1. Sistema control de versiones

El control de versiones se gestionará de manera distribuida utilizando **Git**. Esta herramienta será la encargada de mantener el historial local de los cambios en el código fuente y en la documentación técnica.

5.2. Repositorio central

Como plataforma de alojamiento y colaboración principal, se utilizará **GitHub**. Se mantendrán un solo repositorio para los 2 componentes de arquitectura del sistema (backend y frontend) además de la descripción del proyecto *README.md*.

5.3. Entornos de ejecución y despliegue

- **Entorno Cliente:** El software está diseñado para ejecutarse en navegadores web modernos (Google Chrome, Mozilla Firefox, Microsoft Edge).
- **Paridad de Entornos y Hosting:** Para el despliegue final, se utilizará una plataforma como servicio (**Railway**) para el backend, (**Railway**) para la base de datos y (**Vercel**) para el frontend.

5.4. Stack tecnológico

- **Frontend:** Desarrollado en **JavaScript** utilizando la biblioteca **React**. Las dependencias y versiones de paquetes externos serán gestionadas estrictamente a través del archivo **package.json** utilizando **npm**.
- **Backend (Servidor):** Desarrollado en **Python** utilizando el framework **FastApi**. Las dependencias del servidor se controlarán fijando las versiones exactas en el archivo **requirements.txt** utilizando **pip**.

5.5. Gestión de base de datos

La persistencia de datos y sus cambios estructurales son elementos críticos del sistema. Se utilizará **MySQL** como sistema de gestión de bases de datos relacionales.

5.6. Documentación Integrado

Para asegurar que la documentación técnica y de gestión evolucione a la par del sistema, se adoptará una estrategia de **documentación integrada directamente en el repositorio del proyecto**.

5.7. Comunicación y Reportes de Estado

Para asegurar que todos los interesados (desarrolladores, SCMR y dirección) estén alineados, se establecen dos niveles de comunicación:

- Se utilizará **Jira** como la herramienta oficial para el control del estado. Cada asignación de tareas y reportes de progreso se registran aquí para garantizar la trazabilidad y auditoría del software.
- Se define **WhatsApp** como canal principal para comunicaciones rápidas, alertas urgentes y coordinación logística entre los miembros del equipo.

Referencias

INACAL (2016). NTP-ISO/IEC 12207:2016 Ingeniería de Software y Sistemas. Procesos del ciclo de vida del software.

Zeballos, Y. (2009). *K2BIM: Plan de Configuración Versión 0.9*. Calaméo. Recuperado de <https://www.calameo.com/read/004372137e7b3f511aa5f>